

Guía paso a paso · v2

Guía paso a paso · versión 2

De una secuencia sin etiqueta a un ranking de ligandos

Evolución y Diseño de Proteínas 2027-1 · Proteína TeuB

Cómo usar esta guía

Esta guía es **autosuficiente**: todo lo que necesitas (la secuencia, los parámetros, el código de los scripts, los resultados esperados y los pasos en cada página web) está aquí. No hace falta abrir ningún otro documento.

Cada tipo de instrucción tiene su propio símbolo:

Símbolo	Significa
CONCEPTO	Teoría: lo que necesitas entender antes de hacer el paso
EN LA WEB	Vas a una página web. Los pasos van numerados, clic por clic
EN COLAB	Vas a Google Colab, que corre en una GPU de Google
EN LA TERMINAL	Comandos para tu computadora (Linux). Es opcional: casi todo tiene una ruta web
ANOTA	Un número o una observación que va a tu cuaderno y al entregable
VERIFICA	Un chequeo que atrapa errores antes de que se propaguen
CUIDADO	Un error real que se cometió en esta práctica y cómo evitarlo
LO QUE DEBES OBTENER	Los resultados reales de TeuB, para que compares con los tuyos

Dos rutas posibles. Casi todas las etapas se pueden hacer:

- **por la web** (/): solo necesitas un navegador y una cuenta de Google;
- **por la terminal** (): instalando los programas en tu computadora, que es lo que se hizo en la versión resuelta de esta práctica.

Los resultados son los mismos por las dos rutas. Si es tu primera vez, **usa la ruta web**.

Antes de empezar, abre un documento de notas (Word, Google Docs o un cuaderno de papel). Lo llamaremos **tu cuaderno**. Cada vez que veas , escribe ahí el dato. Al final, el entregable se arma casi solo a partir de tu cuaderno.

Índice

- **Parte 0** · El problema y la biología que necesitas
- **Parte 1** · Preparación
- **Etapas 1** · La secuencia
- **Etapas 2** · BLAST: ¿a qué familia pertenece?
- **Etapas 3** · AlphaFold2: predecir la estructura
- **Etapas 4** · El péptido señal: dónde empieza la proteína madura
- **Etapas 5** · Foldseek: parientes por forma, y el panel de ligandos
- **Etapas 6** · Conseguir y verificar los ligandos
- **Etapas 7** · Acoplamiento molecular (docking)
- **Etapas 8** · Co-plegamiento con Boltz-2
- **Etapas 9** · El ranking: ruido, empates y comparación
- **Etapas 10** · La predicción y el entregable
- **Apéndices** · Errores frecuentes, glosario, lista de verificación, código completo

PARTE 0 · El problema y la biología que necesitas

0.1 · La pregunta

Te dan **una secuencia de 362 aminoácidos**, sin nombre y sin función conocida. Se llama TeuB (número de acceso AGB73230.1) y viene de una bacteria del grupo de *Rhizobium*, bacterias del suelo que viven en las raíces de las plantas.

¿Qué molécula pequeña, qué azúcar, une esta proteína?

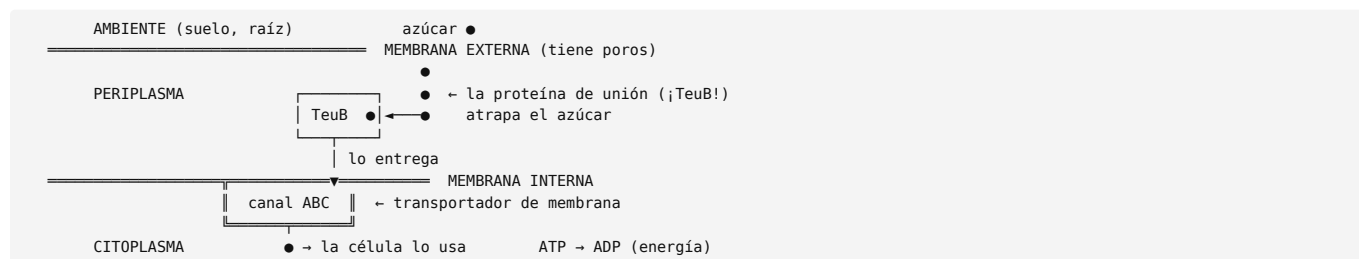
Nadie conoce la respuesta experimental. La práctica **no califica que aciertes**: califica que **razones con evidencia** y que **midas cuánto puedes confiar en tus propios números**.

0.2 · Lo que vas a entregar al final

1. Una LISTA de ligandos ordenada por "afinidad aparente", obtenida por DOS métodos independientes:
 - Docking (AutoDock Vina) → número en kcal/mol
 - Co-plegamiento (Boltz-2) → número ipTM (0 a 1)
2. El PISO DE RUIDO de cada método: cuánto cambia el número si repites el cálculo.
3. Una COMPARACIÓN: ¿los dos métodos están de acuerdo?
4. Un VEREDICTO honesto: ¿se puede saber qué azúcar prefiere?

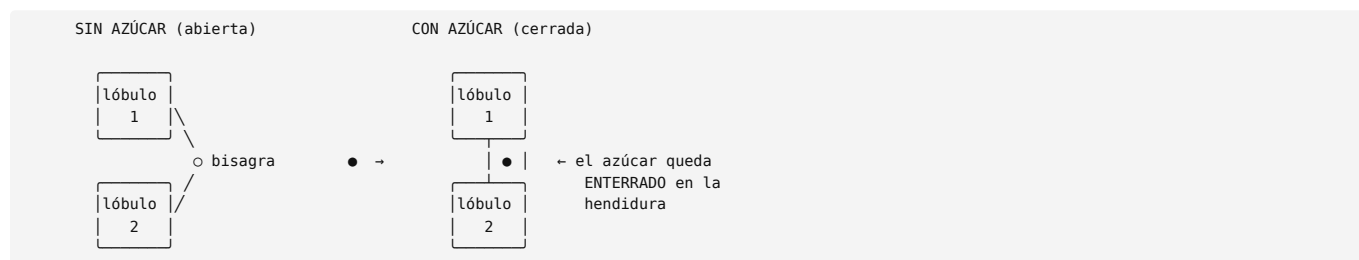
0.3 · 🧠 CONCEPTO: las proteínas periplásmicas de unión a sustrato

Las bacterias como *Rhizobium* tienen **dos membranas** (son Gram-negativas). El espacio entre las dos se llama **periplasma**. Para comer azúcares del ambiente usan un sistema de tres piezas llamado **transportador ABC**:



TeuB sería la **proteína de unión**. Su trabajo es atrapar **un azúcar específico** en el periplasma y entregárselo al canal.

¿Cómo lo atrapa? Estas proteínas tienen **dos lóbulos** unidos por una bisagra, como una almeja o una planta carnívora (en inglés se le dice el mecanismo *Venus flytrap*):



El azúcar se une en la **hendidura** entre los dos lóbulos. Unos 8 residuos de la proteína lo sujetan con **puentes de hidrógeno** con sus grupos OH. En TeuB esos residuos son:

Y38 · S43 · H45 · R174 · W197 · D222 · E248 · C269 (Y = tirosina, S = serina, H = histidina, R = arginina, W = triptófano, D = aspartato, E = glutamato, C = cisteína; el número es la posición en la cadena madura. Esto se explica en la Etapa 4.)

0.4 · 🧠 CONCEPTO: por qué es tan difícil saber qué azúcar une

Problema 1: la familia es "promiscua". Todas estas proteínas tienen **el mismo pliegue**, pero cada una une un azúcar distinto: ribosa, glucosa, galactosa, arabinosa, xilosa, apiosa, alosa... Basta cambiar 2 o 3 residuos del sitio para que cambie el azúcar. Por eso, saber que TeuB pertenece a la familia **no** te dice qué une.

Problema 2: los candidatos son casi la misma molécula. Mira estas cuatro:

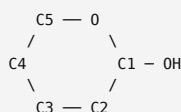
ribosa C5H10O5	xilosa C5H10O5	arabinosa C5H10O5	apiosa C5H10O5	← ¡MISMA FÓRMULA!
-------------------	-------------------	----------------------	-------------------	-------------------

Tienen los mismos átomos. Lo único que cambia es **hacia dónde apunta cada grupo OH** o **cómo se cierra el anillo**. Un método computacional tiene que distinguir diferencias de energía de fracciones de kcal/mol.

Mini-curso de química de azúcares (lo mínimo)

a) El anillo puede ser de 6 o de 5. Un azúcar en agua se cierra sobre sí mismo formando un anillo:

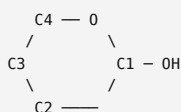
PIRANOSA (anillo de 6: 5 C + 1 O)



ej. β -D-ribopiranos (RIP)

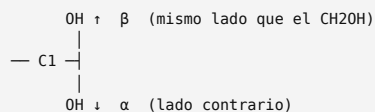
→ el MISMO azúcar puede existir en las dos formas

FURANOSA (anillo de 5: 4 C + 1 O)



ej. β -D-ribofuranosa (BDR)

b) El carbono anomérico (C1) tiene dos orientaciones posibles: α y β . Al cerrarse el anillo, el OH del C1 puede quedar “abajo” (α) o “arriba” (β). Son moléculas distintas, con códigos distintos en el PDB. Por ejemplo, **XYP** = β -D-xilopiranos y **XIP** = α -D-xilopiranos.

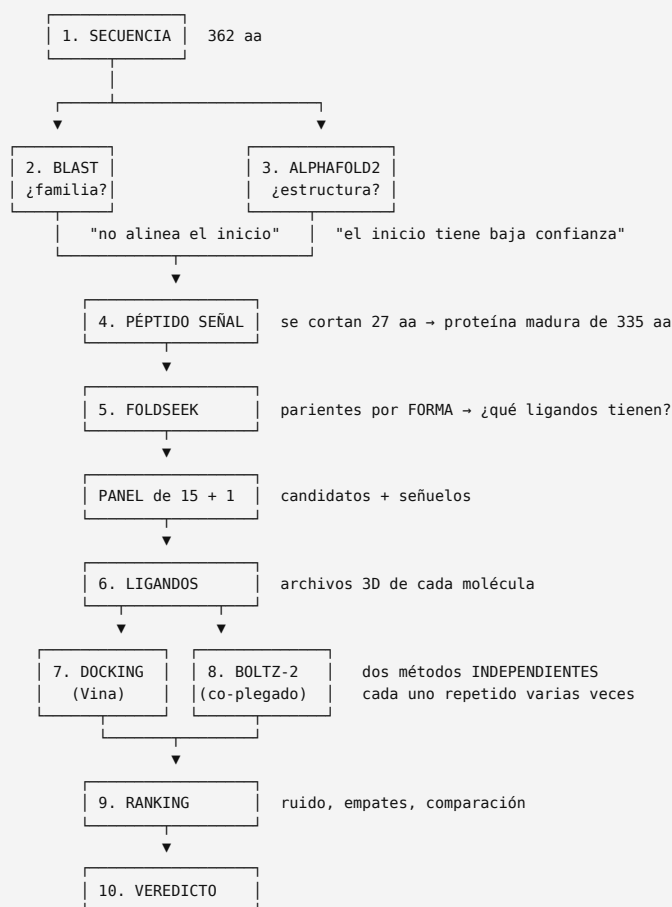


c) Aldosa o cetosa. En una **aldosa** (glucosa, ribosa...) el anillo se cierra en el C1. En una **cetosa** (fructosa, psicosa) se cierra en el C2. Pueden tener la misma fórmula.

d) Los que no son azúcares. Un **ciclitol**, como el inositol, es un anillo de 6 carbonos sin oxígeno en el anillo. Un **poliol**, como el alitol o el treitol, es una cadena abierta con muchos OH. Una **base púrica**, como la hipoxantina, es plana y aromática. En esta práctica sirven como **señuelos**: un buen método debería decir que no se unen.

0.5 · El plan completo, en un dibujo

Cada etapa produce el insumo de la siguiente:



PARTE 1 · Preparación

1.1 · Lo que necesitas

Necesitas	Para qué	Cómo conseguirlo
Un navegador (Chrome o Firefox)	todas las etapas web	ya lo tienes
Una cuenta de Google	usar Google Colab (etapas 3 y 8)	gmail.com
Un programa para ver estructuras 3D (opcional pero muy útil)	ver la proteína y los ligandos	Mol* en la web (https://molstar.org/viewer/), o PyMOL / ChimeraX instalados
Una hoja de cálculo	el análisis de la etapa 9	Google Sheets o Excel
Una carpeta en tu computadora	guardar todo	ver abajo

1.2 · Organiza tus carpetas

Crea **una carpeta para la práctica** y dentro **una subcarpeta por etapa**. Parece burocracia, pero así cada archivo tiene un lugar y siempre sabes en qué punto vas:

```
practica_teub/  
├─ 01_blast/           resultados de BLAST  
├─ 02_structura/       el modelo de AlphaFold (ZIP descomprimido)  
├─ 03_maduro/          el modelo recortado (sin péptido señal)  
├─ 04_foldseek/        la tabla de aciertos y ligandos  
├─ 05_ligandos/        los archivos de cada azúcar  
├─ 06_docking/         resultados del docking  
├─ 07_coplegamiento/   los ZIP de Boltz-2  
└─ 08_analisis/        la hoja de cálculo y el entregable
```

■ **EN LA TERMINAL** (opcional):

```
mkdir -p practica_teub/{01_blast,02_structura,03_maduro,04_foldseek,05_ligandos,06_docking,07_coplegamiento,08_analisis}
```

1.3 · ■ Ruta terminal: instalar los programas (opcional)

Solo si quieres hacer las etapas en tu computadora en lugar de la web. Necesitas Linux o macOS con **miniforge** (<https://github.com/conda-forge/miniforge>):

```
mamba install -y -c conda-forge -c bioconda blast foldseek openbabel vina \  
numpy pandas scipy biopython
```

Nota sobre los comandos de esta guía: todos se escriben desde **dentro de la carpeta** `practica_teub/`. Donde dice python, se refiere al Python de miniforge, que tiene numpy y Biopython.

ETAPA 1 · La secuencia

🧠 CONCEPTO: por qué tanto cuidado con algo tan simple

Una secuencia es una cadena de letras, una por aminoácido. Si se pierde **una sola letra**, todo lo que viene después cambia de posición, y un error de copia se propaga a **todas** las etapas. En esta práctica pasó dos veces: una estructura se predijo con la secuencia equivocada y cuatro corridas de Boltz se hicieron con la secuencia **cortada a la mitad**. Por eso, desde el principio: **longitud, primeras letras y últimas letras**, siempre.

La secuencia de TeuB (362 aminoácidos)

Copia esto en un archivo de texto llamado `teuB.fasta` dentro de `01_blast/`:

```
>TeuB_AGB73230.1  
MKRRFTLQTGSALIAAGAFGIPGILRADDITALLPDTWPSEGENPVVEAGKFAKAGPWKIG  
HSHYGLAGSTHTYQTAFEAEYEISRNKARVADYQFRSADLNASKQVADIEDLIAQKVDATII  
IAPLTTGSAVEGIRKAKAAGIPTVVYLGVRVDTFTVQVGDDFYFGRVMAQFLVDKLGK  
GKVVWLRGVAGHPIDADRYAGAMEVFSKSLQITSTQHGWSYEDSKKIAESLYLSDPDVA  
GWTGDANMSLGLVDALQEAGASTIPPITGEALNGWMRRWNDEKLSSIGPICPPALSTAAL  
RASFALLEGKPIQRNWTNRPKPIVDENLAQFYRSDLTDAYWAPTEMPNEKLLEYFKA
```

El formato **FASTA** tiene una línea que empieza con `>` (el nombre) y, debajo, la secuencia.

Esta es la misma secuencia **en una sola línea**, para pegar en formularios web:

```
MKRRFTLQTGSALIAAGAFGIPGILRADDITALLPDTWPSEGENPVVEAGKFAKAGPWKIGHSHYGLAGSTHTYQTAFEAEYEISRNKARVADYQFRSADLNASKQVADIEDLIAQKVDATIIAPLTTGSAVEGIRKAKAAGIPTVVYI
```

✓ **VERIFICA** (hazlo siempre que pegues una secuencia en cualquier sitio):

Chequeo	Valor correcto
Longitud	362
Empieza con	MKRRTFLQTG
Termina con	NEKLLEYFKA

🌐 Cómo contar letras sin programar:

1. Ve a https://www.bioinformatics.org/sms2/protein_stats.html.
2. Pega la secuencia.
3. Presiona **Submit**. Te dice la longitud.

💻 En la terminal:

```
grep -v '>' 01_blast/teuB.fasta | tr -d '\n' | wc -c # debe dar 362
```

ETAPA 2 · BLAST: ¿a qué familia pertenece?

🧠 CONCEPTO: alinear secuencias

BLAST busca en una base de datos las secuencias que **se parecen** a la tuya y las **alinea**, es decir, las pone una sobre otra para que coincidan el mayor número posible de posiciones. Este es un fragmento **real** del alineamiento de TeuB con su mejor acierto, RbsB de *Bacillus subtilis*:

```
TeuB  102  NASKQVADIEDLIAQKVDAIIIAPLTTGSAVEGIRKAKAAGIPTVVYLRVDTEDFTVQV  161
      ++SKQ +D+EDLI Q VDA++I P  + +  +  A A G+P V  +  V
RbsB   78  DSSKQTSQVDELIIQQGVDALLINPTDSSAISTAVESANAVGVPVVTIDRSAEQGKVETLV  137
```

La línea del medio se lee así:

- **Una letra** (D, L, I...): el aminoácido es **idéntico** en las dos secuencias.
- **+**: los aminoácidos son distintos, pero **parecidos** químicamente, por ejemplo I y L (los dos hidrofóbicos) o D y E (los dos ácidos).
- **Un espacio**: son distintos y no se parecen.

Los números a los lados son **posiciones**. Este fragmento empieza en el residuo 102 de TeuB y en el 78 de RbsB.

Los números que da BLAST, explicados

Número	Qué es	Ejemplo con el acierto #1
% identidad	idénticos + posiciones alineadas	63 idénticos de 221 = 28.5 %
Cobertura (Query Cover)	qué porcentaje de TU secuencia quedó alineado	61 % : el 39 % restante no alinea
E-value	cuántos aciertos así de buenos esperarías por puro azar en una base de ese tamaño	7.7×10^{-20} : prácticamente imposible que sea casual
q.start / q.end	dónde empieza y termina el alineamiento en tu secuencia	empieza en el residuo 102

El E-value con una analogía. Imagina que buscas tu apellido en la guía telefónica de una ciudad. Si es “García”, vas a encontrar miles por azar. Si es “Xochipilli-Brandenburg”, encontrar uno ya significa algo. El E-value dice cuántos “Garcías” esperarías por casualidad. **E < 0.001 se considera significativo; E = 10^{-20} es parentesco seguro.**

🧠 CONCEPTO: la zona crepuscular

Que dos proteínas sean parientes (tienen un ancestro común) **no garantiza** que hagan lo mismo. Lo que puedes concluir depende de la identidad:



TeuB cae **justo en la zona crepuscular** con todos sus parientes de función conocida. Esa es la raíz de toda la dificultad.

🧠 CONCEPTO: dos bases de datos, dos preguntas distintas

Base	Tamaño	Calidad	Para qué se usa aquí
Swiss-Prot	~570 mil proteínas	revisada a mano por expertos; la función tiene respaldo	saber qué hacen los parientes
nr (no redundante)	cientos de millones	automática, casi sin revisar	contar cuántos parientes hay y de qué organismos

⚠ CUIDADO con la anotación circular. En nr vas a ver parientes al 97-100 % anotados como “*ribose transport protein*”. **No es evidencia nueva.** Un programa les puso ese nombre automáticamente porque se parecen a RbsB, que es lo mismo que ya viste en Swiss-Prot. Contarlas como evidencia sería contar el mismo dato dos veces.

 EN LA WEB · 2a. BLAST contra Swiss-Prot

1. Ve a <https://blast.ncbi.nlm.nih.gov/Blast.cgi>.
2. Haz clic en el recuadro **“Protein BLAST”** (protein → protein).
3. En el recuadro **“Enter accession number(s), gi(s), or FASTA sequence(s)”**, pega la secuencia de 362 letras.
4. En **“Job Title”** escribe `TeuB_swissprot`.
5. Baja a **“Choose Search Set”** → en **“Database”** elige `UniProtKB/Swiss-Prot (swissprot)`.
6. En **“Program Selection”** deja marcado `blastp (protein-protein BLAST)`.
7. Presiona el botón azul **“BLAST”** (abajo a la izquierda). Espera de 30 segundos a 2 minutos; la página se actualiza sola.
8. Cuando salga la tabla **“Descriptions”**, observa las columnas **Description, Scientific Name, Query Cover, E value y Per. Ident.**
9. Para guardar los resultados: arriba de la tabla, **“Download”** → **“Hit Table (csv)”**. Guarda el archivo en `01_blast/`. Ese CSV tiene, en orden: *query, subject, % identity, alignment length, mismatches, gap opens, q. start, q. end, s. start, s. end, evalue, bit score*.
10. Para ver dónde alinea cada acierto en tu secuencia, abre la pestaña **“Graphic Summary”**. Las barras de colores muestran qué parte de tu proteína cubre cada acierto. **Fíjate si alguna barra cubre el inicio (residuos 1-40).**

Los E-values pueden cambiar un poco respecto a los de esta guía porque la base de datos crece, pero el panorama debe ser el mismo.

 EN LA WEB · 2b. BLAST contra nr (para contar)

1. Repite los pasos 1 a 4 (en “Job Title” pon `TeuB_nr`).
2. En **“Database”** elige `Non-redundant protein sequences (nr)`.
3. Abre **“Algorithm parameters”**, abajo, en letra pequeña → **“Max target sequences”** → elige **500**.
4. Presiona **“BLAST”**. Puede tardar entre 5 y 20 minutos.
5. En la tabla, ordena por **Per. Ident** y cuenta cuántos superan el 40 %.
6. Abre la pestaña **“Taxonomy”**. Ahí ves de qué organismos son los aciertos: ¿cuántos son de *Rhizobium* y géneros cercanos (familia Rhizobiaceae)?

 EN LA TERMINAL (alternativa a 2a y 2b)

```
# 2a · descargar Swiss-Prot ya formateada (~300 MB) y buscar localmente
cd 01_blast
curl -sL -O https://ftp.ncbi.nlm.nih.gov/blast/db/swissprot.tar.gz && tar xzf swissprot.tar.gz
export BLASTDB=$PWD
blastp -query teuB.fasta -db swissprot -evalue 1e-3 -num_threads 8 -max_target_seqs 250 \
      -outfmt "6 sseqid stitle pident length qstart qend evalue bitscore qcovs" -out swissprot_hits.tsv

# 2b · nr en los servidores del NCBI (-remote); tarda 10-30 min
nohub blastp -query teuB.fasta -db nr -remote -evalue 1e-5 -max_target_seqs 500 \
      -outfmt "6 sseqid stitle pident length qstart qend evalue bitscore qcovs" -out nr_hits.tsv &
cd ..
```

`-outfmt "6 ..."` produce una tabla separada por tabuladores con las columnas que tú eliges.

Qué mirar: cinco preguntas

1. ¿Los aciertos son todos de **una misma familia**?
2. ¿Qué **identidad** tienen? ¿Estás en zona crepuscular?
3. ¿Los parientes con identidad parecida **unen lo mismo** o cosas distintas?
4. ¿Dónde **empiezan** los alineamientos en tu secuencia? ¿Alguno cubre el inicio?
5. ¿Hay parientes del **mismo género** que *TeuB (Rhizobium)*?

 LO QUE DEBES OBTENER

Swiss-Prot: 17 aciertos con E < 0.001, todos proteínas periplásmicas de unión a azúcar (familia RbsB).

#	Proteína	Organismo	% id	q.start	E
1	Ribose import binding protein RbsB	<i>Bacillus subtilis</i>	28.5	102	7.7e-20
2	D-threitol-binding protein	<i>Mycolicibacterium smegmatis</i>	31.5	97	1.3e-19
3	Xylitol-binding protein	<i>M. smegmatis</i>	29.9	88	6.1e-17
4	Ribose import binding protein RbsB	<i>Escherichia coli</i>	23.4	102	3.8e-14
5	Ribose import binding protein RbsB	<i>Salmonella enterica</i>	22.6	102	1.7e-13

#	Proteína	Organismo	% id	q.start	E
6	D-ribose/D-allose-binding protein	<i>Pseudomonas aeruginosa</i>	26.7	100	9.3e-13
7	Galactofuranose-binding protein YtfQ	<i>E. coli</i>	24.8	57	5.3e-11
8	D-apiose import binding protein	<i>Paraburkholderia graminis</i>	24.9	78	3.3e-10
9	D-apiose import binding protein	<i>Rhizobium rhizogenes</i>	28.1	78	8.8e-10
10	D-apiose import binding protein	<i>Actinobacillus succinogenes</i>	26.0	78	6.1e-09
11	D-apiose import binding protein	<i>Rhizobium etli</i>	26.6	78	1.0e-08
12	D-allose-binding protein	<i>E. coli</i>	26.3	106	2.4e-08

 **ANOTA** estas conclusiones:

- **Familia: segura.** Los 17 aciertos son de la misma familia.
- **Ligando: incierto.** Entre los 10 primeros hay **6 ligandos distintos** (ribosa, treitol, xilitol, ribosa/alosa, galactofuranosa y apiosa). El #1 (28.5 %) y el #2 (31.5 %) tienen identidades casi iguales y unen cosas distintas.
- **Zona crepuscular:** todas las identidades están entre 22 y 31 %.
- **El inicio no alinea:** el q.start más bajo es **49** y la mediana es 88. Ningún pariente cubre los primeros 48 residuos. Esto es una **pista para la Etapa 4**.
- **Pista biológica:** dos aciertos de **apiosa** son del género *Rhizobium*, el mismo que TeuB.

nr: 500 aciertos, de los cuales **420 superan el 40 % de identidad**. Unos 260-310, según cómo se cuenten, son de Rhizobiaceae (*Rhizobium*, *Mesorhizobium*, *Sinorhizobium*, *Agrobacterium*...). Los mejores (97-100 %) están anotados automáticamente como “ribose transport...”, lo que **no es evidencia** (anotación circular).

ETAPA 3 · AlphaFold2: predecir la estructura

CONCEPTO: cómo “adivina” AlphaFold la forma de una proteína

AlphaFold2 no simula física. **Aprende de la evolución.** Su idea central es la **coevolución**:

Si dos aminoácidos están **en contacto** en la estructura 3D y uno muta, el otro tiende a mutar también para compensar. Comparando miles de secuencias parientes, se puede detectar qué posiciones “cambian juntas” y, de ahí, cuáles se tocan.

Alineamiento múltiple (MSA) de parientes:			Posiciones 12 y 87 cambian JUNTAS:		
pos:	...12 ...	...87 ...	12	87	
TeuB	... K ...	... E ...	K — E	(+ con -: se atraen)	
pariente A	... K ...	... E ...	K — E		
pariente B	... E ...	... K ...	E — K	(se intercambiaron	
pariente C	... R ...	... D ...	R — D	las cargas, pero	
pariente D	... D ...	... R ...	D — R	siguen complementarias)	
⇒ 12 y 87 probablemente están EN CONTACTO					

Por eso el primer paso de AlphaFold es construir un **MSA** (alineamiento múltiple) con muchos parientes. Más parientes significa más señal. **ColabFold** es una versión de AlphaFold2 que hace la búsqueda del MSA muy rápido con un servidor (MMseqs2) y corre en Google Colab.

CONCEPTO: las dos medidas de confianza

pLDDT (0-100): confianza local, residuo por residuo.

pLDDT	90 — 100		muy alta: la cadena principal y las laterales, bien colocadas
	70 — 90		buenas: el esqueleto es correcto
	50 — 70		baja: tómallo con cuidado
	0 — 50		muy baja: probablemente DESORDENADO, o AlphaFold no sabe

En el archivo PDB que da ColabFold, **el pLDDT está guardado en la columna del B-factor**. Por eso, en un visor 3D, “colorear por B-factor” es colorear por confianza.

PAE (Predicted Aligned Error, en Å): confianza en la posición RELATIVA de dos partes.

La pregunta que responde es: “Si alineo el modelo sobre el residuo *i*, ¿cuánto error espero en el residuo *j*?”. Se dibuja como una matriz. **Un bloque oscuro (PAE bajo) indica un dominio rígido y bien colocado.** Esta es la matriz real de TeuB, promediada por tramos:

Qué mirar

1. Las gráficas PNG (ábre las con doble clic):

- plddt.png: ¿hay una región con la línea muy baja? ¿Dónde está?
- pae.png: ¿ves un cuadro grande oscuro, que es la proteína rígida, y una franja clara, que es algo mal colocado?
- coverage.png: ¿cuántas secuencias hay en el MSA?

2. El modelo en 3D.

EN LA WEB · Ver el modelo en Mol*

1. Ve a <https://molstar.org/viewer/>.
2. Menú de la izquierda **“Download / Open”** → **“Open Files”** → elige el PDB rank_001. También puedes arrastrarlo a la ventana.
3. Para colorear por confianza: en el panel de la derecha, sobre la representación *Polymer*, abre el menú (...) → **“Set Coloring”** → **“Atom Property”** → **“B-factor”** (o **“Validation”** → **“pLDDT”**, según la versión).
4. Mira la forma: **¿ves dos lóbulos con una hendidura en medio?** Ese es el pliegue de la familia. **¿Hay una “cola” suelta de otro color en un extremo?**

3. Los números exactos, opcional con Python. Este script lee el modelo y la matriz PAE:

```
# guarda como p03_plddt_pae.py y corre: python p03_plddt_pae.py 02_estructura/TeuB_full_XXXXX
import sys, glob, json, numpy as np
D = sys.argv[1]
CORTE = 27 # hipotesis: el peptido senal son los residuos 1-27
a3m = glob.glob(f'{D}/*.a3m')[0]
print("MSA:", sum(l.startswith('>') for l in open(a3m)), "secuencias")
pdb = sorted(glob.glob(f'{D}/*rank_001*.pdb'))[0]
bf = {}
for l in open(pdb): # el pLDDT esta en la columna B-factor (caracteres 61-66)
    if l.startswith('ATOM'):
        bf.setdefault(int(l[22:26]), []).append(float(l[60:66]))
p = {k: np.mean(v) for k, v in bf.items()}; N = max(p)
s = np.mean([p[i] for i in range(1, CORTE + 1)]); m = np.mean([p[i] for i in range(CORTE + 1, N + 1)])
print(f"pLDDT medio {np.mean(list(p.values())):.1f}; mínimo {min(p.values()):.1f} en el residuo {min(p, key=p.get)}")
print(f"pLDDT 1-{CORTE} = {s:.1f} vs {CORTE+1}-{N} = {m:.1f}")
pae = np.array(json.load(open(glob.glob(f'{D}/*scores_rank_001*.json')[0]))['pae'])
print(f"PAE senal-vs-madura = {pae[:CORTE, CORTE:].mean():.1f} Å ; dentro de la madura = {pae[CORTE:, CORTE:].mean():.1f} Å")
```

LO QUE DEBES OBTENER

- **MSA: 1841 secuencias.** Es un alineamiento muy profundo, suficiente para una buena predicción.
- **Mejor modelo: pLDDT 92.2, pTM 0.877.** Los 5 modelos quedan entre 91.9 y 92.2: muy consistentes.
- **El inicio tiene muy baja confianza.** Este es el perfil real de pLDDT por tramos de 10 residuos:

```
1- 10 34.0 ██████████
11- 20 35.1 ██████████
21- 30 47.9 ██████████ ← aquí empieza a subir
31- 40 95.8 ██████████
41- 50 96.5 ██████████
... (todo el resto por encima de 89)
351-360 94.7 ██████████
```

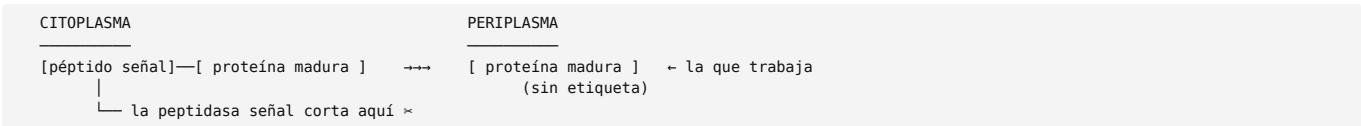
- **pLDDT residuos 1-27 = 36.3, frente a 96.8 en 28-362.** Son 60 puntos de diferencia, con el mínimo (29.1) en el residuo 8.
- **PAE entre los residuos 1-27 y el resto = 29.6 Å:** AlphaFold no sabe dónde va ese tramo. Dentro de la proteína madura, **3.0 Å**.
- **Dos lóbulos,** con un corte natural cerca del residuo 162 y un PAE entre ellos muy bajo: el modelo sabe cómo se orientan uno respecto al otro. Eso indica una **hendidura cerrada**.

 **ANOTA:** “La estructura es muy confiable **excepto los primeros 27 residuos**, que AlphaFold ni pliega ni sabe dónde colocar. Esto coincide con BLAST, que no alinea el inicio.”

ETAPA 4 · El péptido señal: dónde empieza la proteína madura

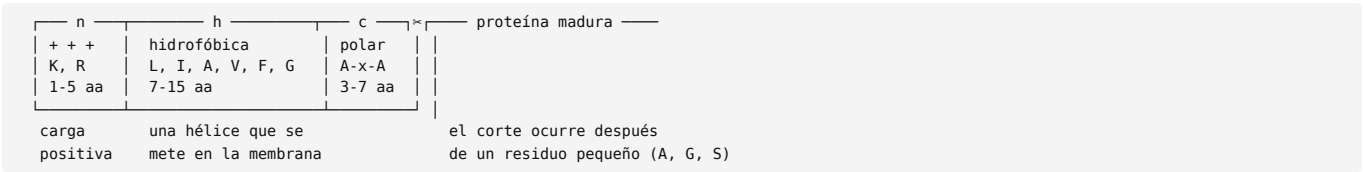
CONCEPTO: una etiqueta de envío que se corta

TeuB se fabrica en el **citoplasma**, pero trabaja en el **periplasma**. Para cruzar la membrana interna lleva en su extremo inicial (el extremo N) un **péptido señal**: una “etiqueta de envío” que la maquinaria de secreción (el sistema **Sec**) reconoce. Al cruzar, una enzima (la **peptidasa señal**) **corta la etiqueta**:



La proteína que une el azúcar es la madura. El péptido señal no forma parte de la estructura funcional, y si lo dejas, estorba en las etapas siguientes.

Cómo se reconoce un péptido señal: tres regiones



CONCEPTO: la hidropatía de Kyte-Doolittle

A cada aminoácido se le asigna un número según cuánto “odia el agua”: **positivo si es hidrofóbico**, como I (+4.5), V (+4.2), L (+3.8) o A (+1.8), y **negativo si es polar o tiene carga**, como R (−4.5), K (−3.9), D (−3.5) o E (−3.5). Si promedias ese número en una **ventana de 9 residuos** que vas deslizando a lo largo de la secuencia, obtienes un **perfil**. Una región h aparece como una **montaña**.

Las tres evidencias que vas a reunir

La práctica pide justificar el corte con **evidencias independientes**. Aquí hay tres:

Evidencia 1 · La secuencia (regiones n/h/c e hidropatía)

EN LA WEB · Perfil de hidropatía con ProtScale

- 1. Ve a <https://web.expasy.org/protscale/>.
- 2. Pega la secuencia de 362 aa en el recuadro.
- 3. En la lista de escalas elige “Hphob. / Kyte & Doolittle”.
- 4. En “Window size” elige 9.
- 5. Presiona “Submit”. Aparece una gráfica: **busca la montaña al principio**, entre los residuos 6 y 22.

EN LA WEB · Segunda opinión con SignalP 6.0 (opcional, pero recomendable)

- 1. Ve a <https://services.healthtech.dtu.dk/services/SignalP-6.0/>.
- 2. Pega la secuencia en formato FASTA.
- 3. En “Organism” elige “Other” (bacterias y arqueas).
- 4. En “Output format” elige “Short output”.
- 5. Presiona “Submit”. Tarda unos minutos.
- 6. Lee el **tipo de péptido señal** que predice (Sec/SPI es el clásico) y **en qué posición pone el corte**. **Compáralo con tu propio análisis**; si difiere, discútelo en el entregable.

EN LA TERMINAL · El mismo análisis en Python:

```
# p04_hidropatia.py -> python p04_hidropatia.py
seq = open('01_blast/teuB_solo_letras.txt').read().strip()
KD = dict(A=1.8, R=-4.5, N=-3.5, D=-3.5, C=2.5, Q=-3.5, E=-3.5, G=-0.4, H=-3.2, I=4.5,
          L=3.8, K=-3.9, M=1.9, F=2.8, P=-1.6, S=-0.8, T=-0.7, W=-0.9, Y=-1.3, V=4.2)
carga = lambda s: sum(c in 'KR' for c in s) - sum(c in 'DE' for c in s)
hid = lambda s: sum(KD[c] for c in s) / len(s)
print(f'n (1-5) {seq[0:5]:<18} carga neta {carga(seq[0:5]):>2f}")
print(f'h (6-22) {seq[5:22]:<18} hidropatia media {hid(seq[5:22]):>2f}")
print(f'c (23-27) {seq[22:27]}")
for i in range(0, 52):
    # ventana deslizante de 9
    v = hid(seq[i:i + 9]); print(f"{i+1:3d}-{i+9:3d} {seq[i:i+9]} {v:>5.2f} {'#' * max(int((v + 4.5) * 4), 0)}")
```

El perfil real de TeuB (ventana de 9; cada # es altura en la escala):

1-9	MKRRTFLQT -1.03 #####	← región n: cargas + (K, R, R)
5-13	TFLLQTGSAL +0.68 #####	
9-17	TGSALIAAG +1.27 #####	
13-21	LIAAGAFGI +2.24 #####	← CIMA de la montaña (región h)
17-25	GAFGIPGIL +1.62 #####	
21-29	IPGILRADD +0.12 #####	← cae: entran R, D, D
25-33	LRADDTIAL +0.39 #####	← ya estamos en la proteína madura
29-37	DTIALLPDT +0.43 #####	

Región	Residuos	Secuencia	Dato
n	1-5	MKRRT	carga neta +3 (K, R, R)
h	6-22	FLQTGSALIAAGAFGIP	hidropatía media +1.27, máximo +2.24 en 13-21
c	23-27	GILRA	termina en A (alanina, un residuo pequeño) ≈
madura	28-362	DDTIALLPDT...	335 aa; empieza con dos aspartatos (D, con carga −)

Evidencia 2 · El pLDDT (ya la tienes de la Etapa 3)

Los residuos 1-27 tienen pLDDT **36.3** y del 28 en adelante **96.8**. La confianza **salta** justo donde termina el péptido.

Evidencia 3 · BLAST (ya la tienes de la Etapa 2)

Ningún pariente alinea antes del residuo **49**.

¿Por qué 49 y no 28? Porque los primeros ~20 residuos de la cadena madura son un extremo variable que no se conserva entre parientes. Eso **no** los hace parte del péptido señal. El pLDDT sí distingue los dos casos: los residuos 28-48 tienen confianza alta, así que forman parte de la proteína plegada.

 **NOTA:** “Corte entre los residuos 27 y 28 (después de GILRA). Tres evidencias: regiones n/h/c con hidropatía, salto de pLDDT de 36 a 97 y ausencia de alineamiento de BLAST en el inicio. Cadena madura de 335 aa, que empieza en DDTIA.”

CONCEPTO: la numeración (esto evita muchísimos errores)

Al cortar 27 residuos, **cada posición cambia de número**. Hay que elegir **una** numeración y no cambiarla nunca:

```
precursor (362):  M K R R T ... G I L R A | D D T I A L L ... Y65 ... C296 ... F K A
posición:        1 2 3 4 5 ...23 24 25 26 27|28 29 30 31 32 33 34 ...           ... 362

madura (335):      | D D T I A L L ... Y38 ... C269 ... F K A
posición:          | 1 2 3 4 5 6 7 ...           ... 335

REGLA:  posición madura = posición del precursor - 27
```

Aquí se usa **siempre la numeración madura, del 1 al 335**. Así, los residuos del sitio son **Y38 S43 H45 R174 W197 D222 E248 C269**.

Cómo quitar el péptido del modelo 3D

Hay dos opciones:

- **(a) Volver a predecir** la estructura en Colab, solo con la secuencia madura de 335 aa. Es lo más limpio.
- **(b) Recortar el PDB** que ya tienes, borrando los residuos 1-27 y renumerando. Es más rápido. **Costo a declarar:** el resto de la proteína se plegó en presencia del péptido.

Aquí se eligió **(b)**.

EN LA WEB · Recortar y renumerar con PDB-Tools Web

1. Ve a <https://wenmr.science.uu.nl/pdbtools/>.
2. Sube el PDB rank_001 de la Etapa 3.
3. Agrega la herramienta **pdb_delres**, que borra residuos, con el rango **1:27**.
4. Agrega después **pdb_reres**, que renumera, empezando en **1**.
5. Ejecuta la cadena y **descarga** el PDB resultante. Guárdalo como 03_maduro/TeuB_maduro_1-335.pdb.

Si prefieres la opción (a), repite la Etapa 3 con la secuencia madura. La tienes abajo.

EN LA TERMINAL · Recortar con Python:

```
# p04_recortar.py -> python p04_recortar.py
import glob
src = glob.glob('02_estructura/*/*rank_001*.pdb')[0]
CORTE = 27
with open('03_maduro/TeuB_maduro_1-335.pdb', 'w') as o:
    for l in open(src):
        if l.startswith(('ATOM', 'TER')) and int(l[22:26]) > CORTE:
            o.write(l[:22] + f"{int(l[22:26]) - CORTE:4d}" + l[26:]) # columnas 23-26 = numero de residuo
        o.write('END\n')
```

La secuencia madura (335 aa), que necesitarás en la Etapa 8. Guárdala como 03_maduro/teuB_maduro_solo_letras.txt:

```
DDTIALLPDTPWSEGENPVVEAGKFAGKAGPWKIGHSHYGLAGSTHTYQTAFEAHEYISRNKARVADYQFRSADLNASKQVADIEDLIAQKVDAAIIAPLTTGSAVEGIRKAKAAGIPTVVYLGRVDTEDFTVQVQGGDFYFGRVMAQFI
```

✓ **VERIFICA:** que tenga **335** letras, empiece en DDTIALLPDT y termine en PNEKLLEYFKA. Luego abre el PDB recortado en Mol* y comprueba que el residuo 38 sea una tirosina (TYR), el 174 una arginina (ARG) y el 269 una cisteína (CYS).

⚠ **CUIDADO: el modelo de respaldo de la práctica tiene un error.** La página reparte un modelo ya hecho (TeuB_modelo_apo.pdb) por si tu predicción falla. Tiene **334 residuos en vez de 335**. En la posición 256 la secuencia real dice ...LNGWMRRWNDEK..., y el respaldo dice ...LNGWMRRWNDEK...: **le falta una arginina**. Por eso, todo lo que viene después del residuo 256 queda corrido en uno. El guion llama “Cys268” a la cisteína del sitio; en la numeración correcta es **C269**. **Lección: compara siempre residuo por residuo los datos que te den.**

ETAPA 5 · Foldseek: parientes por forma, y el panel de ligandos

CONCEPTO: la estructura se conserva más que la secuencia

A lo largo de la evolución, la secuencia cambia rápido, pero **la forma cambia muy lento**, porque la forma es lo que hace funcionar a la proteína. Dos proteínas pueden tener solo 20 % de identidad y ser casi superponibles:

identidad de SECUENCIA entre TeuB y sus parientes:	similitud de ESTRUCTURA entre TeuB y sus parientes:
 20 % "casi irreconocibles"	 TM-score 0.8-0.9 "casi la misma forma"

Foldseek busca parientes **por forma**. Su truco es traducir cada estructura 3D a un "alfabeto estructural" de 20 letras, llamado **3Di**, donde cada letra describe **cómo está orientado un residuo respecto a sus vecinos**. Así, comparar estructuras se vuelve tan rápido como comparar secuencias.

CONCEPTO: el TM-score

El **TM-score** (0 a 1) mide qué tanto se parecen dos formas completas:

TM-score	< 0.3		formas no relacionadas (al azar)
	0.5		MISMO PLIEGUE (umbral clásico)
	> 0.8		casi superponibles

CONCEPTO: por qué Foldseek sirve para encontrar LIGANDOS

Foldseek busca en el **PDB**, el banco mundial de estructuras **experimentales**. Muchas se cristalizaron **con su ligando dentro**. Cada pariente con ligando es **un experimento de unión que alguien ya hizo**:

TeuB (sin ligando)	—Foldseek→	2IOY	proteína de unión de Thermoanaerobacter	+ RIBOSA
		50CP	proteína de unión de Shewanella	+ ARABINOSA
		2GBP	proteína de unión de E. coli	+ GLUCOSA
		3T95	proteína de unión de ...	+ APIOSA
		...		
		↳	de aquí sale la LISTA DE CANDIDATOS	

⚠ La trampa de PDB100

En la página de Foldseek, la base **"PDB100"** agrupa las cadenas con **secuencia idéntica** y muestra **solo una por grupo**. Si una proteína tiene 10 cristales (sin ligando, con glucosa, con galactosa...), ves **uno**, y puede ser el que no tiene nada. Por eso, **cuando un acierto aparezca sin ligando, busca las otras entradas de la misma proteína** (se explica abajo).

🌐 EN LA WEB · Buscar con Foldseek

1. Ve a <https://search.foldseek.com/search>.
2. **Sube el PDB recortado** (TeuB_maduro_1-335.pdb): arrástralo al recuadro o usa **"Upload PDB/mmCIF"**. Usa el modelo recortado, no el completo.
3. En **"Databases"** deja marcado **PDB100**. Opcionalmente marca también **AFDB-SwissProt**.
4. En **"Mode"** deja **"3Di/AA"**.
5. Presiona **"Search"**. Tarda menos de un minuto.
6. En la pestaña **PDB100** verás una tabla con **Target** (código PDB y cadena), **Description**, **Prob.**, **Seq. Id.** (identidad de secuencia), **E-value** y **TM-score** (o *Score*, según la versión).
7. Abre una hoja de cálculo y copia los **40-50 primeros aciertos**: código PDB, descripción, identidad y TM-score.

🌐 EN LA WEB · Averiguar qué ligando tiene cada acierto

Por cada código PDB de tu lista (por ejemplo 2IOY):

1. Ve a <https://www.rcsb.org/structure/2IOY> (cambia el código).
2. Baja hasta la sección **"Small Molecules"**. Ahí aparece cada ligando con su **ID** (código de 3 letras), **nombre** y **fórmula**.
3. **Ignora lo que no es un ligando biológico**: agua (HOH), iones (NA, CL, MG, ZN, CA, K, SO4, PO4...), crioprotectores (**GOL** = glicerol, **EDO** = etilenglicol, **PEG**...), tampones (TRS, EPE, MES...) y detergentes. Se agregaron para cristalizar, no son lo que la proteína une en la naturaleza.
4. Anota en tu hoja: **código PDB → ligando(s) → fórmula**.
5. **Si la entrada no tiene ligando útil** (la trampa de PDB100): en la misma página, en **"Macromolecules"**, haz clic en el enlace de **UniProt** de la proteína. En la página de UniProt, sección **"Structure"**, verás **todas** las entradas del PDB de esa proteína. Revisa las que no viste.
6. Cuando termines, cuenta en tu hoja cuántas veces aparece cada ligando.

💻 EN LA TERMINAL (lo que se hizo aquí): base PDB completa y consulta automática

Con la base completa (6.5 GB) no existe la trampa de PDB100, porque ves **todas** las cadenas.

```
cd 04_foldseek
foldseek databases PDB pdb_db tmp_dl # descarga una sola vez (~10 min)
cd ..
foldseek easy-search 03_maduro/TeuB_maduro_1-335.pdb 04_foldseek/pdb_db 04_foldseek/hits.tsv /tmp/fs \
--format-output "query,target,fident,alnlen,evalue,bits,prob,alntmscore,qcov,tcov,theader" \
--max-seqs 4000 -e 10 --threads 8 # tarda ~8 segundos
```

Para no revisar 150 páginas a mano, se le preguntó al RCSB con su **API GraphQL**, que devuelve los datos en formato de máquina:

```
# p05_rcsb_ligandos.py -> python p05_rcsb_ligandos.py
import json, urllib.request, collections
rows = [l.rstrip('\n').split('\t') for l in open('04_foldseek/hits.tsv')]
ids = []
for r in rows:
    pid = r[1].split('-')[0].split('_')[0].upper()
    if pid not in ids: ids.append(pid)
    if len(ids) == 150: break
Q = '{ entries(entry_ids: %s) { rcsb_id nonpolymer_entities { nonpolymer_comp { chem_comp { id name formula } } } } }'
E = {}
for i in range(0, len(ids), 50):
    req = urllib.request.Request('https://data.rcsb.org/graphql', headers={'Content-Type': 'application/json'},
    data=json.dumps({'query': Q % json.dumps(ids[i:i+50])}).encode())
    for e in json.load(urllib.request.urlopen(req, timeout=90))['data']['entries']:
        if e: E[e['rcsb_id']] = [n['nonpolymer_comp']['chem_comp'] for n in (e['nonpolymer_entities'] or [])]
BASURA = set('H0H S04 P04 G0L E0D PEG PG4 PGE 1PE MPD ACT ACY FMT CL BR IOD NA K MG CA ZN MN FE NI CO CU CD
HG CS N03 TRS EPE MES BTB CIT FLC TAR MLA SCN AZI NH4 IMD BME DTT DMS URE ACE NAG BOG LDA C8E P6G 12P 15P
SIN OXL MAE EOH IPA MRD TLA OCT PE4 B3P CAC NH2 UNX UNL'.split())
cnt, info, donde, apo = collections.Counter(), {}, collections.defaultdict(list), []
for pid, ligs in E.items():
    buenos = [l for l in ligs if l['id'] not in BASURA]
    if not buenos: apo.append(pid)
    for l in buenos: cnt[l['id']] += 1; info[l['id']] = l; donde[l['id']].append(pid)
print(f"entradas: {len(E)} sin ligando: {len(apo)}")
for c, n in cnt.most_common(25):
    print(f"{c:<5}{n:>3} {info[c]['formula']:<14}{info[c]['name'][:40]:<41} {''.join(donde[c][:5])}")
```

LO QUE DEBES OBTENER

- **Muchos parientes por forma:** 2811 aciertos con la base completa. Los mejores tienen **TM-score de 0.76 a 0.92** y probabilidad 1.00, pero **solo 17-25 % de identidad de secuencia**. Foldseek encuentra parientes justo donde BLAST ya no resuelve: BLAST dio 17 aciertos.
- De las **150 primeras entradas únicas, 67 (45 %) no tienen ligando** (son estructuras “apo”) y 83 sí.
- **Ligandos más frecuentes:**

Código	Veces	Fórmula	Nombre	Algunas entradas
BGC	13	C6H12O6	β-D-glucopiranososa	2GBP, 3GBP, 2IPL, 4RXM...
RIP	9	C5H10O5	β-D-ribopiranososa	1DBP, 2IOY, 2GX6, 4ZJP...
GAL	7	C6H12O6	β-D-galactopiranososa	2RJO, 8ABP, 4WWH...
HPA	6	C5H4N4O	hipoxantina	1JFT, 1VPW, 1QPZ...
XYP	4	C5H10O5	β-D-xilopiranososa	3C6Q, 4YWH, 5XSS...
INS	4	C6H12O6	<i>mio</i> -inositol	4IRX, 4YO7, 4RU1...
PAV	4	C5H10O5	D-apiosa	3EJW, 3T95, 4PZ0, 6DSP
ARA	2	C5H10O5	α-L-arabinopiranososa	1BAP, 4RXT
FRU	2	C6H12O6	β-D-fructofuranosa	2X7X, 3BRQ

- **La dificultad, cuantificada:** en la lista aparecen **11 ligandos distintos con fórmula C5H10O5** y **10 con C6H12O6**.

 **ANOTA** la tabla de ligandos, las fórmulas repetidas y el porcentaje de entradas apo.

Construir el panel: la parte que requiere criterio

No basta con tomar los más frecuentes. Que la glucosa aparezca 13 veces dice qué proteínas se han cristalizado más, no qué une TeuB. El panel se armó con **cuatro criterios explícitos**, que tienes que poder defender:

Criterio	Ligandos elegidos	Qué pregunta responde
(a) Frecuencia en Foldseek	BGC glucosa, RIP ribosa, GAL galactosa, XYP xilosa	lo que más se ha visto en la familia
(b) Convergencia BLAST + Foldseek	PAV D-apiosa	la única que aparece en los dos, y con parientes de <i>Rhizobium</i>
(c) Pares con la MISMA fórmula	ARA / AHR: arabinosa, anillo de 6 / de 5	¿el método distingue el tamaño del anillo?
	RIP / BDR: ribosa, anillo de 6 / de 5	ídem
	GAL / GZL: galactosa, anillo de 6 / de 5	ídem
	ALL / WEB: alosa (aldosa) / psicosa (cetosa)	¿distingue una aldosa de una cetosa?
	BGC / FRU: glucosa (aldosa) / fructosa (cetosa)	ídem

Criterio	Ligandos elegidos	Qué pregunta responde
(d) Señuelos , de fácil a difícil de rechazar	INS <i>mio</i> -inositol: ciclitol, misma fórmula que la glucosa	¿lo confunde con un azúcar?
	X9X D-alitol: poliol de cadena abierta	¿rechaza algo sin anillo?
	HPA hipoxantina: base púrica, ni azúcar ni poliol	¿rechaza algo químicamente distinto?

Además se agregó **3VB** (D-treitol, C4H10O4), que viene del panel de respaldo de la práctica. En total son **16 ligandos**.

 **ANOTA** de qué entradas del PDB sale cada ligando (lo pide el entregable): AHR de 5OCP, BDR de 3KSM, ALL de 5DTE, WEB de 8WLB, GZL de 2VK2, X9X de 4WT7 y 3VB de 4RSM; los demás, en la tabla de arriba.

ETAPA 6 · Conseguir y verificar los ligandos

CONCEPTO: el diccionario químico del PDB (CCD)

Cada molécula pequeña del PDB tiene una **ficha** en el *Chemical Component Dictionary* (**CCD**), identificada por un **código de 3 caracteres** (BGC, RIP, XYP...). La ficha define átomos, enlaces y estereoquímica, e incluye unas **coordenadas "ideales"**: una forma 3D limpia, generada por computadora, **con todos los hidrógenos**. Es el punto de partida estándar para el docking.

CONCEPTO: formatos de archivo (qué hay adentro)

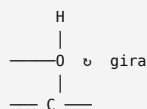
SDF: átomos con coordenadas y, después, los enlaces. Este es el inicio real de RIP_ideal.sdf:

```
RIP                                     ← nombre
CCTOOLS-1004241127                   ← programa que lo generó
                                     ← (línea de comentario)
20 20 0 0 1 0 0 0 0999 V2000         ← 20 átomos, 20 enlaces
0.5380 0.2920 -1.2190 C 0 0 ...       ← átomo 1: x, y, z, elemento
-0.6510 -0.5750 -0.8030 C 0 0 ...     ← átomo 2
...
```

PDBQT (el formato de AutoDock Vina) es un PDB con dos agregados: el **tipo de átomo** en la última columna y un **árbol de torsiones**, que indica qué enlaces pueden girar. Este es un fragmento real de RIP.pdbqt:

```
ROOT                                     ← la parte RÍGIDA: el anillo
ATOM 1 C UNL 1 0.538 0.292 -1.219 ... C ← carbono
ATOM 6 O UNL 1 1.686 -0.061 -0.450 ... OA ← O "aceptor" de puente H
ENDROOT
BRANCH 1 7                               ← un enlace que PUEDE GIRAR (C1-O1)
ATOM 7 O UNL 1 0.812 0.086 -2.606 ... OA
ATOM 8 H UNL 1 1.563 0.653 -2.829 ... HD ← H "donador" (el del OH)
ENDBRANCH 1 7
...
TORSDOF 4                                ← 4 torsiones activas en total
```

Cada OH es una "rama" que gira:



La ribopiranososa tiene 4 OH que giran → 4 torsiones.
Una furanosa suele tener UNA MÁS, porque su CH2OH
queda fuera del anillo y agrega un enlace libre.
Esto va a importar en la Etapa 9.

EN LA WEB · Descargar cada ligando

Por cada código del panel (ejemplo con BGC):

1. Ve a <https://www.rcsb.org/ligand/BGC> (cambia el código).
2. Revisa en la ficha el **nombre** y la **fórmula**. ¿Es la molécula que esperabas? ¿Es α o β ?
3. Busca **"Download"** / **"Download Files"** → **"Ideal Coordinates (SDF)"**.
4. Guárdalo como 05_ligandos/BGC_ideal.sdf.

Descarga directa, sin navegar por la página: https://files.rcsb.org/ligands/download/BGC_ideal.sdf (cambia el código).

Los 16 códigos del panel:

BGC RIP GAL XYP PAV INS ARA AHR BDR ALL WEB FRU GZL X9X HPA 3VB

Para SwissDock (la ruta web de la Etapa 7) puedes subir el SDF o el mol2 directamente, o pegar el **SMILES**, sin convertir nada. La conversión a PDBQT solo hace falta para Vina en la terminal.

EN LA TERMINAL · Descargar y convertir los 16 de una vez:

```
cd 05_ligandos
for c in BGC RIP GAL XYP PAV INS ARA AHR BDR ALL WEB FRU GZL X9X HPA 3VB; do
  curl -s -o ${c}_ideal.sdf https://files.rcsb.org/ligands/download/${c}_ideal.sdf
  obabel -isdf ${c}_ideal.sdf -omol2 -o ${c}.mol2      # para SwissDock / visualizar
  obabel -isdf ${c}_ideal.sdf -opdbqt -o ${c}.pdbqt    # para Vina: tipos de átomo + torsiones
done
cd ..
```

Al convertir a PDBQT, Open Babel asigna los tipos de átomo, **conserva solo los hidrógenos polares** (los de los OH, que forman puentes de hidrógeno) y construye el árbol de torsiones.

✓ VERIFICA: los tres chequeos del guion

Por cada ligando:

Chequeo	Por qué importa	Cómo se ve
1. La fórmula del archivo = la fórmula de la ficha	así sabes que bajaste la molécula correcta	cuenta los C, H y O en el SDF; la línea 4 dice cuántos átomos hay
2. Tiene hidrógenos	sin H, los puentes de hidrógeno se calculan mal	en el SDF aparecen átomos H
3. No es plano	un SDF "2D" (todas las z = 0) hace que el docking parta de una forma absurda	la tercera coordenada no es 0.0000 en todos los átomos

La excepción es la **hipoxantina (HPA)**, que **debe** ser plana porque es un anillo aromático.

Chequeo automático:

```
# p06_verificar_ligandos.py -> python p06_verificar_ligandos.py
import collections, numpy as np
for c in 'BGC RIP GAL XYP PAV INS ARA AHR BDR ALL WEB FRU GZL X9X HPA 3VB'.split():
  L = open(f'05_ligandos/{c}_ideal.sdf').read().splitlines()
  n = int(L[3][:3]); at = L[4:4 + n]
  el = collections.Counter(l[31:34].strip() for l in at)
  xyz = np.array([float(l[0:10]), float(l[10:20]), float(l[20:30])] for l in at)
  X = xyz - xyz.mean(0); grosor = 2 * np.sqrt(np.linalg.eigvalsh(X.T @ X / n)[0])
  tors = open(f'05_ligandos/{c}.pdbqt').read().count('BRANCH') // 2
  formula = ''.join(f"{e}{el[e]}" for e in 'CHNO' if el[e])
  print(f"{c:<4} {formula:<10} H={el['H']:<3} torsiones={tors:<3} grosor={grosor:.2f} Å",
        "2D!" if np.allclose(xyz[:, 2], 0) else ("plano (aromatico)" if grosor < 0.3 else "3D ok"))
```

🔗 LO QUE DEBES OBTENER

Los 16 pasan los tres chequeos. HPA sale plana, que es lo correcto. Fíjate en las **torsiones**:

Par (misma fórmula)	Anillo de 6	Anillo de 5
arabinosa	ARA: 4	AHR: 5
ribosa	RIP: 4	BDR: 5
galactosa	GAL: 6	GZL: 7

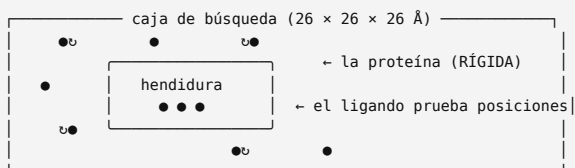
📝 **NOTA:** "Cada furanosa tiene una torsión activa más que su piranosa." Te va a servir para explicar un resultado de la Etapa 9.

ETAPA 7 · Acoplamiento molecular (docking)

🧠 CONCEPTO: qué hace un programa de docking

El docking intenta responder: "Si meto esta molécula en este bolsillo, ¿en qué posición queda mejor, y qué tan bien?". Tiene dos partes:

1. BÚSQUEDA: probar miles de posiciones, giros y torsiones del ligando dentro de una CAJA

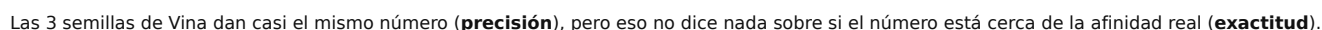


2. PUNTAJE: a cada posición le calcula un número (función de puntaje de Vina)

puntaje = contactos estéricos + puentes de H + hidrofobicidad
+ penalización por cada torsión que el ligando "congela"
→ en kcal/mol; MÁS NEGATIVO = "MEJOR"

1. **La proteína es rígida.** Vina no la mueve. Si la hendidura está cerrada, como en TeuB, el ligando tiene que caber en el hueco que ya existe.
2. **La búsqueda es aleatoria.** Depende de una **semilla** (*seed*), que es el número con el que arranca el generador aleatorio. Con otra semilla, el resultado puede cambiar un poco. **Repetir con varias semillas mide el ruido de la búsqueda.**
3. **El puntaje NO es la afinidad real.** El error típico de Vina frente a mediciones de laboratorio es de ~ 2 kcal/mol.

Esta distinción es central para el entregable:



⚠ **CUIDADO:** el guion da un centro (3.4 / 4.0 / 1.3) que corresponde al **modelo de respaldo**. Si lo usas con tu modelo, la caja puede quedar en el vacío. El centro de TeuB en el modelo propio es **x = 3.38, y = -3.17, z = 0.79**, y se calculó como el promedio de los átomos de cadena lateral de los 8 residuos del sitio.

1. Abre `TeuB_maduro_1-335.pdb` con un editor de texto (Bloc de notas, gedit...).
2. Busca las líneas ATOM de los residuos **38, 43, 45, 174, 197, 222, 248 y 269**. El número de residuo va en las columnas 23-26, después de la letra de cadena A. Por ejemplo:

3. Copia a la hoja de cálculo las coordenadas x, y, z de **todos los átomos de cadena lateral** de esos 8 residuos, es decir, **todos menos N, CA, C y O**. Son 44 átomos.
4. Calcula el promedio de cada columna: =PROMEDIO(B2:B45). **Ese es el centro.**
5. Para el tamaño: toma el máximo menos el mínimo de cada columna (unos 17 Å como máximo) y súmale ~8 Å de margen. Resulta una caja de **26 Å** por lado.

```
# p07_centro_caja.py -> python p07_centro_caja.py
import numpy as np
SITIO = [38, 43, 45, 174, 197, 222, 248, 269]
P = np.array([[float(l[30:38]), float(l[38:46]), float(l[46:54])]]
              for l in open('03_maduro/TeuB_maduro_1-335.pdb')
              if l.startswith('ATOM') and int(l[22:26]) in SITIO
              and l[12:16].strip() not in ('N', 'CA', 'C', 'O'))
print(f"len(P): {len(P)} atoms; centro = {P.mean(0).round(2)}; extension = {(P.max(0) - P.min(0)).round(1)} Å")
```

SwissDock usa por dentro **el mismo motor que AutoDock Vina**. Los nombres de los botones pueden variar un poco según la versión de la página.

1. Ve a <https://www.swissdock.ch/>.
2. **Ligando:** sube el archivo .mol2 o .sdf del ligando, o pega su SMILES. Presiona el botón para **preparar el ligando** (*Prepare ligand*).
3. **Blanco (target):** sube TeuB_maduro_1-335.pdb, **el recortado**, y presiona **preparar el blanco** (*Prepare target*).
4. **Parámetros:**
 - o motor: **AutoDock Vina**;
 - o **centro de la caja** (*box center*): $x = 3.38$, $y = -3.17$, $z = 0.79$, o el que calculaste para **tu** modelo;
 - o **tamaño de la caja** (*box size*): 26, 26, 26;
 - o **exhaustividad** (*sampling exhaustivity*): la más alta que permita la página. Anótala y **usa la misma en todos los ligandos**.

- Envía el trabajo y guarda el **ID** o el enlace de resultados.
- En los resultados, anota el **mejor puntaje (Vina score, kcal/mol)** y mira la pose en el visor: ¿quedó **dentro de la hendidura**?
- Repite con otra semilla, o simplemente otra vez**, al menos para algunos ligandos. Así mides el ruido. Si la página no deja elegir semilla, corre el mismo ligando 2 o 3 veces.
- Repite para todos los ligandos del panel con **exactamente los mismos parámetros**.

EN LA TERMINAL · Docking con AutoDock Vina (lo que se hizo aquí)

Paso 1, preparar el receptor.

```
cd 06_docking
obabel -ipdb ../03_maduro/TeuB_maduro_1-335.pdb -opdbqt -o receptor.pdbqt -xr -p 7.4
```

- xr: receptor **rígido**.
- p 7.4: agrega los **hidrógenos que corresponden a pH 7.4**, el pH fisiológico: His, Asp, Glu y Lys con la carga que tendrían en la célula.

Paso 2, correr Vina con un ligando y una semilla:

```
vina --receptor receptor.pdbqt --ligand ../05_ligandos/BGC.pdbqt \
--center_x 3.38 --center_y -3.17 --center_z 0.79 \
--size_x 26 --size_y 26 --size_z 26 \
--exhaustiveness 32 --num_modes 9 --seed 101 --cpu 8 \
--out out_BGC_s101.pdbqt > log_BGC_s101.txt
```

Opción	Qué hace	Valor usado
--center_*, --size_*	la caja	centro del sitio; 26 Å
--exhaustiveness	cuánto busca (el valor por omisión es 8)	32 : 4 veces más búsqueda, resultados más estables
--num_modes	cuántas poses guarda	9
--seed	la semilla aleatoria	101, 202 y 303 (3 réplicas)
--cpu	núcleos	8

Paso 3, todos los ligandos, con un script **reanudable** (si se interrumpe, al volver a correrlo salta las corridas que ya tienen resultado):

```
cat > run.sh <<'SH'
#!/bin/bash
for c in BGC RIP GAL XYP PAV INS ARA AHR BDR ALL WEB FRU GZL X9X HPA 3VB; do
  for seed in 101 202 303; do
    o=out_${c}_s${seed}.pdbqt
    [ -s "$o" ] && continue
    vina --receptor receptor.pdbqt --ligand ../05_ligandos/${c}.pdbqt \
--center_x 3.38 --center_y -3.17 --center_z 0.79 --size_x 26 --size_y 26 --size_z 26 \
--exhaustiveness 32 --num_modes 9 --seed $seed --cpu 8 --out $o > log_${c}_s${seed}.txt 2>&1
  done
done
SH
chmod +x run.sh
tmux new -s docking # abre una sesión que sobrevive si cierras la terminal
./run.sh # 48 corridas x ~3 min cada una
# Ctrl+B y luego D = salir sin detenerlo; tmux attach -t docking = volver
```

⚠ CUIDADO: nos pasó. El script se lanzó **dos veces** sin detener la primera, y quedaron dos copias compitiendo por la CPU. **Antes de relanzar, revisa con** `pgrep -af run.sh`.

Cómo leer el resultado

Cada `log_*.txt` termina con una tabla. Esta es la real de XYP con la semilla 101:

```
mode | affinity | dist from best mode
      | (kcal/mol) | rmsd l.b. | rmsd u.b.
-----+-----+-----+-----
1      -6.406      0          0          ← LA MEJOR POSE: este es el número del ranking
2      -5.907      1.269      3.58      ← otras poses, peores
3      -5.724      0.9626     3.045
4      -5.636      0.6794     3.207
```

- affinity**: el puntaje de esa pose.
- rmsd l.b. / u.b.**: qué tan lejos está esa pose de la mejor, en Å.

Para extraer el mejor puntaje de todos los logs (el -a es necesario porque la barra de progreso hace que grep crea que el archivo es binario):

```
for f in log_*_s*.txt; do echo "$f $(grep -a -A3 '-----' $f | sed -n 2p | awk '{print $2}'); done
```

✓ VERIFICA: ¿la pose quedó en el bolsillo?

Un buen puntaje en la superficie de la proteína no significa nada.

🌐 En Mol*: abre `TeuB_maduro_1-335.pdb` y `out_XYP_s101.pdbqt` en la misma ventana. El primer modelo del PDBQT es la mejor pose. ¿El ligando está **dentro de la hendidura**, rodeado por Y38, S43, H45, R174, W197, D222, E248 y C269? Para verlos: pestaña de selección → escribe 38 43 45 174 197 222 248 269.

🐍 En Python: mide la distancia del centro del ligando al centro del sitio y cuántos de los 8 residuos quedan a menos de 4 Å:

```
import re, numpy as np
SITIO = [38, 43, 45, 174, 197, 222, 248, 269]
rec = [(int(l[22:26]), np.array([float(l[30:38]), float(l[38:46]), float(l[46:54])]))
for l in open('03_maduro/TeuB_maduro_1-335.pdb') if l.startswith('ATOM')]
centro = np.array([3.38, -3.17, 0.79])
txt = open('06_docking/out_XYP_s101.pdbqt').read().split('ENDMDL')[0] # solo el modo 1
L = np.array([float(l[30:38]), float(l[38:46]), float(l[46:54]) for l in txt.splitlines() if l.startswith('ATOM')])
cerca = {r for r, x in rec if r in SITIO and np.linalg.norm(L - x, axis=1).min() < 4}
print(f"distancia al centro {np.linalg.norm(L.mean(0) - centro):.1f} Å; residuos del sitio en contacto: {len(cerca)}/8")
```

🎯 LO QUE DEBES OBTENER

Puesto	Ligando	Semilla 101	202	303	Media	DE
1	XYP xilopiranososa	-6.406	-6.416	-6.401	-6.41	0.008
2	ARA arabinopiranososa	-6.151	-6.131	-6.152	-6.14	0.012
3	HPA hipoxantina 🐼	-6.104	-6.097	-6.104	-6.10	0.004
4	INS inositol 🐼	-6.093	-6.093	-6.087	-6.09	0.004
5	GAL galactopiranososa	-6.050	-6.060	-6.038	-6.05	0.011
6	RIP ribopiranososa	-6.000	-6.000	-6.008	-6.00	0.005
7	ALL alopiranososa	-5.945	-5.952	-5.950	-5.95	0.004
8	GZL galactofuranosa	-5.775	-5.790	-5.783	-5.78	0.008
9	PAV apiosa	-5.747	-5.743	-5.764	-5.75	0.011
10	FRU fructofuranosa	-5.725	-5.723	-5.727	-5.73	0.002
11	BDR ribofuranosa	-5.707	-5.706	-5.711	-5.71	0.003
12	BGC glucopiranososa	-5.630	-5.665	-5.647	-5.65	0.018
13	AHR arabinofuranosa	-5.506	-5.500	-5.508	-5.50	0.004
14	WEB psicopiranososa	-5.456	-5.457	-5.453	-5.46	0.002
15	X9X alitol 🐼	-5.227	-5.221	-5.237	-5.23	0.008
16	3VB treitol	-4.975	-4.970	-4.976	-4.97	0.003

🐼 = señuelo; DE = desviación estándar de las 3 semillas

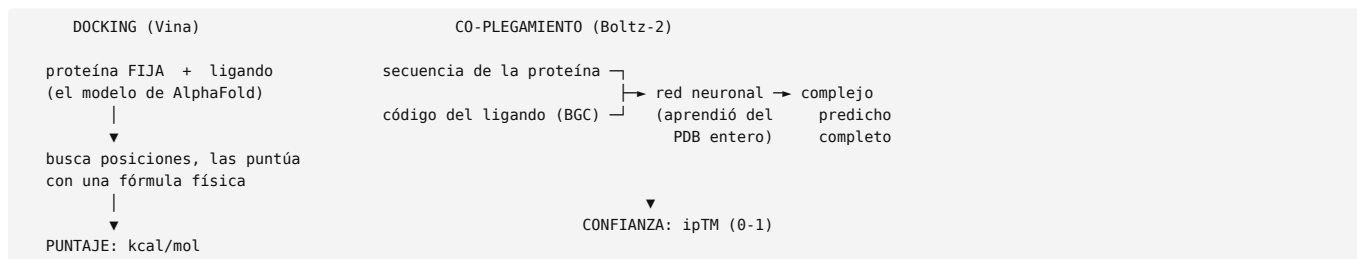
📝 ANOTA:

- **Todas** las mejores poses están **dentro del bolsillo**: $a \leq 1.5$ Å del centro y tocando 7 u 8 de los 8 residuos.
- Las 3 semillas difieren **como mucho en 0.035 kcal/mol**. La búsqueda es muy reproducible.
- **Dos señuelos (HPA e INS) quedan en el 3.º y 4.º lugar**, por encima de casi todos los azúcares.
- Todo el panel cabe en **1.43 kcal/mol**, menos que el error típico de Vina (~2 kcal/mol).

ETAPA 8 · Co-plegamiento con Boltz-2

🧠 CONCEPTO: un enfoque completamente distinto

Boltz-2 es una red neuronal de la familia de **AlphaFold 3**. No hay función física ni proteína rígida: le das la **secuencia** de la proteína y el **código** del ligando, y **predice el complejo entero a la vez**. La proteína “se pliega alrededor” del ligando.



Cómo genera cada modelo (difusión). Parte de átomos colocados al azar, como una nube, y los va “limpiando” paso a paso hasta obtener una estructura. Cada punto de partida al azar produce un modelo un poco distinto. Con **3 semillas × 5 muestras = 15 modelos** por ligando, la dispersión

entre esos 15 es el **piso de ruido del método 2**.

 CONCEPTO: las confianzas de Boltz

Número	Qué mide	Rango
ipTM	confianza en la interfaz proteína-ligando: ¿está bien colocado el ligando respecto a la proteína?	0-1; más alto es mejor
pTM	confianza en la estructura global	0-1
ranking_score	combinación que Boltz usa para ordenar sus propios modelos	0-1
has_clash	¿hay átomos que chocan?	debe ser 0

 **Otra vez, sin plantillas.** Si el servidor le da a Boltz un cristal como plantilla, puede **copiar** la pose del ligando del cristal, y entonces el ipTM mide memoria, no predicción.

 EN COLAB · Correr Boltz-2 (una corrida por ligando)

- Abre https://colab.research.google.com/github/sokrypton/ColabFold/blob/main/ColabFold2_preview.ipynb.
- Entorno de ejecución → Cambiar tipo de entorno → GPU T4 → Guardar.**
- En la **celda de instalación** elige `model = boltz2` y ejecútala con el botón ▶ de la celda. La primera vez descarga los pesos del modelo y tarda unos minutos.
- En la **celda de entrada** llena:

Campo	Valor	¡Ojo!
protein	la secuencia MADURA de 335 aa (Etapa 4)	no las 362
ligand_ccd	el código, por ejemplo BGC	uno por corrida; el mismo código que usaste en el docking
ligand_smiles	vacío	el código ya trae la molécula
jobname	teub_bgc	cambia según el ligando
msa_mode	mmseqs2_server	
seeds	1,2,3	
plantillas	desactivadas	control de contaminación
num_recycles, num_diffusion_samples	por omisión	dan 5 muestras por semilla

- Ejecuta las celdas de predicción. Tarda de 5 a 10 minutos.
- Descarga el ZIP que genera (`teub_bgc_xxxxx.result.zip`). Si no se descarga solo, búscalo en el panel de archivos 📁 → clic derecho → Descargar.
- Repite los pasos 4 a 6 para cada ligando** cambiando solo `ligand_ccd` y `jobname`. No hace falta reinstalar.

 **VERIFICA ANTES DE PEGAR la secuencia:** que termine en `..PNEKLEYFKA`. Si puedes, agrega una celda nueva con `print(len(protein))`: debe dar **335**.

 **CUIDADO: nos pasó dos veces.**

- En **4 corridas** la secuencia quedó **cortada en 132 aa** al copiarla. Faltaba medio sitio de unión, el ipTM bajó a ~0.75 y hubo que repetirlas.
- En **una corrida** se escribió XIP (α-xilosa) en vez de XYP (β-xilosa, la del docking). Son moléculas distintas, así que no se pueden comparar.

Qué hay en cada ZIP y cómo revisarlo

```
af3_output/teub_bgc_xxxxx/
├─ teub_bgc_xxxxx_data.json          ← LA ENTRADA REAL: qué secuencia, qué ligando,
│                                   qué semillas, si hubo plantillas ← ¡REVÍSALO!
├─ teub_bgc_xxxxx_summary_confidences.json ← iptm, ptm, has_clash del MEJOR modelo
├─ teub_bgc_xxxxx_ranking_scores.csv   ← ranking_score de los 15 modelos (seed, sample)
├─ teub_bgc_xxxxx_model.cif           ← el mejor modelo (cadena A = proteína, B = ligando)
└─ seed-1_sample-0/ ... seed-3_sample-4/ ← los 15 modelos, cada uno con su cif y confianzas
```

 **Revisión sin programar** (con el navegador y una hoja de cálculo):

- Descomprime el ZIP.
- Abre `__data.json` arrastrándolo a Firefox o Chrome, que muestran los JSON de forma legible. Comprueba:
 - `sequences → protein → sequence`: ¿empieza en DDTIA y termina en EYFKA?
 - `sequences → ligand → ccdCodes`: ¿es el código correcto?
 - `modelSeeds`: ¿es [1, 2, 3]?
 - `templates`: ¿está vacío ([])?
- Abre `__summary_confidences.json` y anota `iptm` y `has_clash`.
- Abre `__ranking_scores.csv` en tu hoja de cálculo. Son 15 filas: calcula la **media** (=PROMEDIO) y la **desviación estándar** (=DESVEST). Ese es el ruido del método para ese ligando.

5. Abre `_model.cif` en Mol*. ¿El ligando (cadena B) está **en la hendidura**, entre los dos lóbulos?

■ **Revisión automática de todos los ZIP** (valida los parámetros y calcula el ipTM medio de los 15 modelos):

```
# revisar_boltz.py -> python revisar_boltz.py (busca los ZIP en 07_coplegamiento/)
import zipfile, json, glob, numpy as np
madura = open('03_maduro/teuB_maduro_solo_letras.txt').read().strip()
for z in sorted(glob.glob('07_coplegamiento/teub_*.result.zip')):
    zf = zipfile.ZipFile(z); nombres = zf.namelist()
    d = json.load(zf.open([n for n in nombres if n.endswith('_data.json')][0]))
    prot = [s['protein'] for s in d['sequences'] if 'protein' in s][0]
    lig = [s['ligand']['ccdCodes'][0] for s in d['sequences'] if 'ligand' in s][0]
    ok = prot['sequence'] == madura and not prot.get('templates') and d['modelSeeds'] == [1, 2, 3]
    ip = [json.load(zf.open(n))['iptm'] for n in nombres if 'seed-' in n and n.endswith('summary_confidences.json')]
    print(f"'OK ' if ok else 'MAL' {lig:4s} {len(prot['sequence'])} aa ipTM = {np.mean(ip):.3f} ± {np.std(ip, ddof=1):.3f} (n={len(ip)})")
```

⚠ Hallazgo: Boltz-2 borra el átomo “O1”

Al comparar los átomos del modelo con los de la ficha del CCD, se descubrió que **en los 10 ligandos que tienen un átomo llamado O1, ese átomo desaparece**:

```
Ficha CCD de BGC (glucosa): C1 C2 C3 C4 C5 C6 O1 O2 O3 O4 O5 O6 = 12 átomos pesados
Modelo de Boltz-2:         C1 C2 C3 C4 C5 C6 O2 O3 O4 O5 O6 = 11 ← ¡falta O1!
                        ^
                        |
                        el OH del carbono anomérico
```

- **Afectados:** RIP, YYP, ARA, AHR, BDR, BGC, GAL, ALL y GZL (pierden el OH anomérico) y FRU (pierde el O del CH₂OH).
- **Completo:** INS, X9X, HPA, 3VB, WEB y PAV, porque ninguno tiene un átomo llamado O1.
- **Consecuencia:** para las aldosas, Boltz modela un **residuo glicosilo** (como si el azúcar formara parte de una cadena de azúcares), **no el azúcar libre**, y **no puede distinguir α de β**. Hay que **declararlo en el entregable**.

Lección: cuenta los átomos. En Mol*, selecciona la cadena B y mira cuántos átomos tiene.

🎯 LO QUE DEBES OBTENER

Puesto	Ligando	ipTM medio (15 modelos)	DE
1	AHR arabinofuranosa	0.975	0.008
2	BDR ribofuranosa	0.973	0.012
3	GZL galactofuranosa	0.963	0.007
4	YYP xilopiranosas	0.961	0.003
4	RIP ribopiranosas	0.961	0.003
4	ARA arabinopiranosas	0.961	0.003
7	HPA hipoxantina 🍄	0.961	0.007
8	ALL alopiranosas	0.957	0.010
9	BGC glucopiranosas	0.957	0.009
10	INS inositol 🍄	0.957	0.006
11	GAL galactopiranosas	0.956	0.011
12	PAV apiosa	0.954	0.008
13	WEB psicopiranosas	0.949	0.007
14	3VB treitol	0.946	0.005
15	FRU fructofuranosa	0.942	0.012
16	X9X alitol 🍄	0.929	0.011

📝 ANOTA:

- **Todos** los ligandos, incluidos los señuelos, tienen ipTM entre 0.93 y 0.98.
- El ligando queda en el sitio en los 15 modelos de cada uno.
- La dispersión entre modelos (0.003-0.012) es **del mismo tamaño** que las diferencias entre ligandos.

ETAPA 9 · El ranking: ruido, empates y comparación

🧠 CONCEPTO: el piso de ruido

Si mides dos veces lo mismo y obtienes números un poco distintos, esa variación es el **ruido**. **Una diferencia entre dos ligandos solo es real si es claramente mayor que el ruido**.

Criterio de empate usado: dos ligandos A y B empatan si

$$\underbrace{|\text{media}_A - \text{media}_B|}_{\text{diferencia}} < \underbrace{2 \times \sqrt{DE_A^2 + DE_B^2}}_{\text{"margen de error" de esa diferencia}}$$

El 2 corresponde, aproximadamente, a un 95 % de confianza.

Ejemplo resuelto 1: Boltz, AHR contra PAV

AHR: 0.975 ± 0.008 PAV: 0.954 ± 0.008

diferencia = $0.975 - 0.954 = 0.021$
margen = $2 \times \sqrt{(0.008^2 + 0.008^2)} = 2 \times \sqrt{(0.000128)} = 2 \times 0.0113 = 0.023$

$0.021 < 0.023 \rightarrow$ EMPATE. No se puede decir que AHR sea mejor que PAV.

Ejemplo resuelto 2: Boltz, AHR contra X9X

AHR: 0.975 ± 0.008 X9X: 0.929 ± 0.011

diferencia = 0.046
margen = $2 \times \sqrt{(0.008^2 + 0.011^2)} = 2 \times 0.0136 = 0.027$

$0.046 > 0.027 \rightarrow$ DISTINTOS. AHR sí supera a X9X.

Ejemplo resuelto 3: Vina, HPA contra INS (el par más cercano)

HPA: -6.102 ± 0.004 INS: -6.091 ± 0.004

diferencia = 0.011 margen = $2 \times \sqrt{(0.004^2 + 0.004^2)} = 0.011$

$\rightarrow$ justo en el límite. Con el ruido de semillas, Vina "separa" a casi todos... pero ese ruido NO incluye el error de la función de puntaje (~ 2 kcal/mol).

CONCEPTO: ¿los dos métodos ordenan igual? (Spearman)

Vina da kcal/mol e ipTM da un número entre 0 y 1: **no se pueden comparar directamente**. Lo que sí se puede comparar son los **puestos**. La **correlación de Spearman (ρ)** mide si dos listas ordenan parecido:

$\rho = +1$ mismo orden exacto
 $\rho = 0$ ninguna relación
 $\rho = -1$ orden invertido

Ejemplo pequeño, con 4 ligandos:

	puesto Vina	puesto Boltz	d = diferencia	d ²
XYP	1	2	-1	1
ARA	2	3	-1	1
AHR	3	1	+2	4
X9X	4	4	0	0
			$\Sigma d^2 =$	6

$\rho = 1 - (6 \times \Sigma d^2) / (n \times (n^2 - 1)) = 1 - (6 \times 6) / (4 \times 15) = 1 - 36/60 = 0.40$

En la hoja de cálculo es más fácil:

- =JERARQUIA.EQV(B2; B\$2:B\$17; 1) da el puesto de Vina (1 = el más negativo).
- =JERARQUIA.EQV(C2; C\$2:C\$17; 0) da el puesto de Boltz (1 = el más alto).
- =COEF.DE.CORREL(D2:D17; E2:E17) sobre las dos columnas de puestos da ρ .

En inglés las funciones se llaman RANK.EQ y CORREL.

CONCEPTO: ¿colocan el ligando en el mismo lugar? (RMSD)

Aunque los puntajes no coincidan, vale la pena ver si los dos métodos **ponen el ligando en la misma posición**. Se superpone la proteína de Boltz sobre la de Vina y se mide la distancia promedio entre los átomos del ligando en las dos poses, el **RMSD**:

RMSD < 2 Å $\rightarrow$ la misma pose, prácticamente
RMSD > 4 Å $\rightarrow$ poses distintas

 En Mol*: abre el model.cif de Boltz y el out_*.pdbqt de Vina junto con el receptor. Usa la herramienta de **superposición** (*Superpose*) sobre la cadena de proteína y compara visualmente dónde queda cada ligando.

El análisis completo en una hoja de cálculo

Arma una hoja con estas columnas:

A: Ligando	B: Vina media	C: Vina DE	D: ipTM media	E: ipTM DE	F: puesto Vina	G: puesto Boltz	H: $\Delta = G - F$
XYP	-6.408	0.008	0.961	0.003	=JERARQUIA.EQV(B2;B\$2:B\$17;1)	=JERARQUIA.EQV(D2;D\$2:D\$17;0)	=G2-F2

Luego:

- **Spearman:** =COEF.DE.CORREL(F2:F17; G2:G17).
- **Empate con el líder de Boltz** (ordena primero por D): =SI(ABS(D\$2-D3) < 2*RAIZ(E\$2^2+E3^2); "empate"; "distinto").

■ Con Python: 08_analisis/comparar.py hace todo esto y además el RMSD entre poses.

LO QUE DEBES OBTENER: la tabla comparativa

Ligando	Puesto Vina	Vina	Puesto Boltz	ipTM	Δ
XYP	1	-6.41	4	0.961	+3
ARA	2	-6.14	4	0.961	+2
HPA 🐼	3	-6.10	7	0.961	+4
INS 🐼	4	-6.09	10	0.957	+6
GAL	5	-6.05	11	0.956	+6
RIP	6	-6.00	4	0.961	-2
ALL	7	-5.95	8	0.957	+1
GZL	8	-5.78	3	0.963	-5
PAV	9	-5.75	12	0.954	+3
FRU	10	-5.73	15	0.942	+5
BDR	11	-5.71	2	0.973	-9
BGC	12	-5.65	9	0.957	-3
AHR	13	-5.50	1	0.975	-12
WEB	14	-5.46	13	0.949	-1
X9X 🐼	15	-5.23	16	0.929	+1
3VB	16	-4.97	14	0.946	-2

Cómo interpretarla (esto es lo que se discute en el entregable)

1. **Vina:** con el ruido de semillas, los 16 ligandos quedan separados. **Pero** ese ruido no incluye el error de la función (~2 kcal/mol), que es mayor que todo el rango (1.43). Vina es **preciso pero no necesariamente exacto** (recuerda el dibujo de la diana).
2. **Boltz:** los **12 primeros empatan** con el líder. Solo WEB, 3VB, FRU y X9X se separan, y hacia abajo. Solo 15 de 120 pares superan el piso de ruido.
3. **Los métodos casi no coinciden:** Spearman $\rho = 0.39$ ($p = 0.14$), una correlación débil y no significativa.
4. **El mayor desacuerdo está en las furanosas.** AHR pasa del 13.º al 1.º lugar y BDR del 11.º al 2.º. Una explicación posible: **Vina penaliza cada torsión activa**, y cada furanosa tiene una más que su piranosa (lo anotaste en la Etapa 6). Boltz no tiene esa penalización.
5. **Pero colocan el ligando en el MISMO sitio:** las poses de Vina y Boltz difieren en **1-2 Å de RMSD. El desacuerdo está en el puntaje, no en la geometría.**
6. **Señuelos:** ninguno de los dos rechaza a HPA ni a INS. Solo coinciden en poner al poliol **X9X al fondo**.

ETAPA 10 · La predicción y el entregable

Cómo formular la predicción (se escribe ANTES de ver los números)

La práctica pide escribir qué ligando esperas que gane, **basándote solo en las Etapas 2-5**. Este es el método:

Paso 1, lista a los candidatos con evidencia:

Candidato	Evidencia a favor	Debilidad
Ribosa (RIP)	acierto #1 de BLAST; 2.º más frecuente en Foldseek	28.5 %, zona crepuscular; el #2 une otra cosa
Glucosa (BGC)	la más frecuente en Foldseek (13)	la frecuencia refleja qué se cristaliza más, no qué une TeuB
Apiosa (PAV)	Swiss-Prot (#8-#11, 2 de Rhizobium) y Foldseek (4)	identidad también baja (~26 %)

Paso 2, pesa cada evidencia con tres preguntas:

- ¿Es fuerte por sí sola? En zona crepuscular, no.
- ¿Es independiente de las demás? Las anotaciones de nr, por ejemplo, no lo son.
- ¿**Varias fuentes independientes** apuntan a lo mismo? Eso es lo que más pesa.

Paso 3, agrega el contexto biológico: ¿qué azúcares hay en el ambiente del organismo? *Rhizobium* vive en las raíces, y la **apiosa es un azúcar de la pared celular de las plantas**.

Predicción resultante: D-*apiosa* (PAV), con la ribosa como segunda opción.

Honestidad: si ya viste los números del docking antes de escribir la predicción, **dilo**. Vale más declararlo que esconderlo.

Resultado: PAV quedó 9.^º en Vina y 12.^º en Boltz, dentro del empate. **Ningún método la confirma ni la descarta.**

El entregable: tres páginas como máximo

Sección	Qué va	Peso
1. El ranking	los 10 mejores de cada método, con su número (kcal/mol; ipTM)	30 %
2. Pisos de ruido	el ruido de cada método, cómo lo mediste y qué diferencias lo superan; empates marcados	20 %
3. Comparación	la tabla con la columna Δ , Spearman y la discusión de los desacuerdos	20 %
4. Construcción del panel	de qué aciertos de Foldseek salió cada ligando y con qué criterio; tu predicción	15 %
5. Cadena de inferencias y péptido señal	tabla de 7 filas (etapa · herramienta · entrada · resultado · qué concluyes y qué no); dónde cortaste y con qué evidencia	10 %
6. Veredicto	una frase, proporcionada a la evidencia	5 %

Plantilla de la tabla de la cadena de inferencias:

Etapas	Herramienta	Entrada	Qué salió	Qué concluyo · qué NO
BLAST	blastp / Swiss-Prot y nr	362 aa	17 aciertos RbsB, 22-31 % id	familia sí · ligando no
AlphaFold2	ColabFold, sin plantillas	362 aa	pLDDT 92, dos lóbulos	estructura confiable · no la función
Péptido señal	n/h/c, pLDDT, BLAST	modelo + secuencia	corte 27/28	madura de 335 aa
Foldseek	PDB	madura 1-335	TM 0.76-0.92 con 17-25 % id	parientes por forma · no el ligando
Ligandos	CCD + Open Babel	16 códigos	SDF/PDBQT verificados	panel listo
Docking	Vina × 3 semillas	receptor + ligandos	−6.41 a −4.97 kcal/mol	reproducible · no es afinidad
Co-plegamiento	Boltz-2 × 15	secuencia + código	ipTM 0.93-0.98	todos caben · no discrimina

El veredicto al que se llegó:

“No: con lo que medí, no puedo decir qué ligando prefiere TeuB.” Los dos métodos colocan todos los ligandos en el mismo bolsillo. Pero Boltz empata a 12 de 16, Vina ordena con una precisión que su función de puntaje no respalda, ninguno rechaza a los señuelos y sus rankings no correlacionan ($p = 0.39$). La hipótesis mejor fundamentada, la apiosa, se apoya en la homología y la biología, no en el cálculo.

Recuerda: acertar no se califica. Se califica que midas, que midas también el ruido y que no afirmes más de lo que tus números sostienen. **Responder “no” con buen argumento vale más que responder “sí” sin él.**

APÉNDICES

A · Los errores de esta práctica: cómo se detectaron y cómo evitarlos

#	Error	Cómo se detectó	Regla
1	AlphaFold se corrió con la secuencia de ejemplo (59 aa)	log.txt: “length 59”	revisa la longitud antes de “Run all”
2	Al modelo de respaldo le falta una Arg en la posición 256	comparación residuo por residuo	compara siempre los datos ajenos con los tuyos
3	El centro de la caja del guion no sirve para tu modelo	corresponde a otras coordenadas	calcula el centro en tu modelo
4	Dos copias del docking a la vez	pgrep -af run.sh	revisa antes de relanzar
5	Boltz con la secuencia cortada a 132 aa	data.json e ipTM bajo	verifica que termine en ...EYFKA
6	XIP en vez de XYP	data.json	el mismo código en los dos métodos
7	Tomar las anotaciones de nr como evidencia	razonamiento	distingue lo experimental de lo inferido
8	Boltz borra el O1	el número de átomos no coincidía	cuenta los átomos

El patrón común: casi todos se atrapan **mirando lo que de verdad entró al programa** (log.txt, config.json, data.json) en lugar de confiar en lo que creías haber puesto.

B · Lista de verificación final

- ☐ Secuencia: 362 aa, empieza en MKRRTFLQTG y termina en NEKLLEYFKA
- ☐ BLAST Swiss-Prot: familia, identidades, q.start mínimo y ligandos del top 10 anotados
- ☐ BLAST nr: cuántos superan el 40 % y cuántos son de Rhizobiaceae
- ☐ AlphaFold: log.txt dice 362, use_templates: false, pLDDT y PAE anotados
- ☐ Péptido señal: tres evidencias; corte 27/28; madura de 335 aa
- ☐ Numeración madura fija; sitio Y38 S43 H45 R174 W197 D222 E248 C269 verificado
- ☐ Foldseek con el modelo **recortado**; tabla de ligandos; fórmulas repetidas
- ☐ Panel con criterios explícitos y entradas de origen
- ☐ **Predicción escrita antes de ver resultados**
- ☐ Ligandos: fórmula, hidrógenos, no planos
- ☐ Docking: el mismo centro, caja y exhaustividad para todos; ≥ 3 réplicas; poses en el bolsillo
- ☐ Boltz: secuencia madura de 335, sin plantillas, semillas 1-3; data.json revisado en cada ZIP
- ☐ Tabla comparativa con Δ , pisos de ruido, empates marcados y Spearman
- ☐ Veredicto proporcionado a la evidencia

C · Glosario

Término	Significado
Anómero α/β	las dos orientaciones del OH del carbono anomérico al cerrarse el anillo
Apo / holo	estructura sin ligando / con ligando
CCD	Chemical Component Dictionary: la ficha de cada ligando del PDB (código de 3 letras)
Cobertura	qué porcentaje de tu secuencia alinea con el acierto
Coevolución	posiciones que mutan juntas porque están en contacto
Docking	colocar un ligando en una proteína rígida y puntuarlo
E-value	cuántos aciertos así de buenos esperarías por azar
Exhaustividad	cuánto busca el docking
Furanosa / piranosa	anillo de 5 / de 6 átomos
ipTM	confianza en la interfaz proteína-ligando (0-1)
MSA	alineamiento múltiple de secuencias
PAE	error esperado en la posición relativa de dos residuos (Å)
PDBQT	formato de Vina: PDB + tipos de átomo + torsiones
Péptido señal	etiqueta de envío en el extremo N; se corta al llegar al destino
Piso de ruido	variación al repetir una medición; una diferencia menor no significa nada
pLDDT	confianza local por residuo (0-100)
Plantilla	estructura conocida que el predictor puede copiar; se desactiva
RMSD	distancia promedio entre dos conjuntos de átomos (Å)
Semilla	número que inicializa lo aleatorio; cambiarla da una réplica
Señuelo	molécula que no debería unirse; sirve de control negativo
Spearman ρ	correlación entre dos ordenamientos
TM-score	similitud de forma global (0-1; más de 0.5 indica el mismo pliegue)
Zona crepuscular	~20-35 % de identidad: se infiere el pliegue, no la función

D · Enlaces usados en esta guía

Eta	Recurso	Dirección
1	Contar letras	https://www.bioinformatics.org/sms2/protein_stats.html
2	BLAST	https://blast.ncbi.nlm.nih.gov/Blast.cgi
3	ColabFold (AlphaFold2)	https://colab.research.google.com/github/sokrypton/ColabFold/blob/main/AlphaFold2.ipynb
3	AlphaFold2-WebGPU	https://martin-steinegger.github.io/alphafold2-webgpu
3-9	Visor Mol*	https://molstar.org/viewer/
4	ProtScale (hidropatía)	https://web.expasy.org/protscale/
4	SignalP 6.0	https://services.healthtech.dtu.dk/services/SignalP-6.0/
4	PDB-Tools Web	https://wenmr.science.uu.nl/pdbtools/
5	Foldseek	https://search.foldseek.com/search
5-6	RCSB PDB	https://www.rcsb.org/ (estructuras: /structure/XXXX; ligandos: /ligand/XXX)
6	SDF ideal directo	https://files.rcsb.org/ligands/download/XXX_ideal.sdf
7	SwissDock	https://www.swissdock.ch/
8	ColabFold2 preview (Boltz-2)	https://colab.research.google.com/github/sokrypton/ColabFold/blob/main/ColabFold2_preview.ipynb

E • Bibliografia

- Altschul et al. (1997) Gapped BLAST and PSI-BLAST. *Nucleic Acids Res.* doi:10.1093/nar/25.17.3389
- Jumper et al. (2021) AlphaFold2. *Nature*. doi:10.1038/s41586-021-03819-2
- Mirdita et al. (2022) ColabFold. *Nat Methods*. doi:10.1038/s41592-022-01488-1
- van Kempen et al. (2024) Foldseek. *Nat Biotechnol*. doi:10.1038/s41587-023-01773-0
- Trott & Olson (2010) AutoDock Vina. *J Comput Chem*. doi:10.1002/jcc.21334
- Eberhardt et al. (2021) AutoDock Vina 1.2.0. *J Chem Inf Model*. doi:10.1021/acs.jcim.1c00203
- Abramson et al. (2024) AlphaFold 3. *Nature*. doi:10.1038/s41586-024-07487-w
- Passaro et al. (2025) Boltz-2. *bioRxiv*. doi:10.1101/2025.06.14.659707